Main goal of course

The main goal of this course is to learn how to model problems instead of solving them. We won't work directly with algorithm, but we will use solvers that use them to solve our models. Find the best way to model a given problem to find an optimal solution in a acceptable time frame.

Constraint Satisfaction Problem

A CSP is made of:

- Set of Variables (V1, V2, V3, ... , Vn)
- Set of Domain of those variables(Dv1, Dv2, Dv3, ... , Dvn)
- Set of Constraints on those variables(Cv1, Cv2, Cv3, ... , Cvn)

Thanks to constraint we can add new conditions to our model instead of rewriting code like in imperative programming.

Constraint Optimization Problem

We have the same three sets like a CSP, but we add a cost function f: D^n -> *A* We look for assignments that either minimize the cost function or maximize it.

Search Space

Given a CSP the search space is the set of n-tuples:

< D1, D2, D3, ... , Dn >

In this set we look for points that satisfy the constraints for the given CSP. This set is commonly represented as a tree, also called search tree.

Relation between CSP and COP

CSP and COP are closely related too each other, there isn't much difference when solving a CSP compared to a COP or vice versa.

Finding a valid solution for a CSP is relatively fast, meanwhile searching the best solution for a COP means traversing the entire search tree.

Most of the times we are happy we good solutions, not necessarily the optimal one.

SAT Solving

A SAT solver is a set of boolean variables X1, ... X2. We have clauses that work on those variables, let's see them as our constraints on those variables.

We can transform some CSP into SAT problems, but it's not always worth it since in SAT solvers we only have cluases and boolean variables we lose higher level of information compared to a CSP.

We need to re-encode those information using boolean variables and clauses this means using more variables, thus increasing the search space.

SAT Solvers, unlike CSP solvers, can learn from errors.

Local search

Allows us to explore the search tree with random choices. We have differenet techniques for this:

- Montecarlo
- Simulated annealing (Temperature method)
- Particle swarm

When choosing a random point on the search space we can explore neighbor nodes.

We can apply local search to CSP

We create a penalty score when we find a wrong solution.

We can remove some constraint so we can find an initial solution and then we can refine it.

Integer Linear Programming

We have constraints that are linear and we also have a linear cost function. We can use branch and bound to find optimal solution

- Branch, because we explore two areas for a solution one will improve the scoring function, meanwhile the other will worsen it.

We can apply Branch and Bound until we find the optimal solution.

Constraint solving

A constraint solver is a procedure capable of transforming a CSP *P* into an equivalent CSP.

A constraint solver can be:

- Complete: if from the CSP *P* it can transform it into an equivalent CSP *P'* in a solved form. A solved form is an implicit description of its solutions
- Incomplete: *P* is transformed into a simpler CSP but its solution are implicit, means we cannot directly determine the values of the solution.

Incomplete solver can be useful when we can accept an approximation for a problem, allowing to find a good solution in less time compared to complete solvers.

Symmetry breaking

For some model we could end up to the same solution following different paths in our search tree. This means that we are going to lose time on finding equivalent solutions. For example in the N-Queen example we could obtain the same result by rotating the matrix. To avoid this we can break symmetry of our model by forcing some values to guide the solver to a specific solution. This requires prior knowledge of the domain of the problem.

Node consistency

A CSP *P* is node consistent if for every variable X in *P*, the unary constraint on X coincides with its domain.

Ex: X >= 0; where X in **N** (natural numbers); this example is node consistent.

Ex: X >= 5; where X in **N** (natural numbers);
this example is NOT node consistent, but it admits solutions, so its considered only consistent.

Ex: X != Y, Y != Z; where X in (0, 0), Y in (0,0) and Z in (0,0) this example is node consistent but not consistent.

Arc consistency

A binary constraint *C* on X and Y with domains Dx and Dy is called arc consistent if: for every value *a* in Dx and *b* in Dy they satisfy the constraint *C*

A CSP is arc consistent if every constraint *C* is arc consistent.

For example: X < Y; X in 5..10, Y in 3..7 this CSP is consistent, it admits at least one solution. But is not arc consistent, since we have some value in Dx and Dy that do not satisfy the constraint.

Arc consistency allows us to reduce the size of the domain through constraint propagation.

Bounds consistency

A binary constraint *C* on variables X and Y with domains Dx and Dy is bound consistent if:

- the min(Dx) can be matched to a value in [min(Dy)..max(Dy)]
- the max(Dx) can be matched to a value in [min(Dy)..max(Dy)]
- the min(Dy) can be matched to a value in [min(Dx)..max(Dx)]
- the max(Dy) can be matched to a value in [min(Dx)..max(Dx)]

The difference with arc consistency is: AC works for every value in the domain meanwhile bounds consistency works only with the bounds of the domain.

Example: 2*X = Y; X in {0, 1, 2}; Y in {0, 4} This CSP is consistent, bound consistent but it's not arc consistent since we can't use the value 1 for X for any value of Y. If we apply AC-1 we would get the following CSP: 2*X = Y; X in {0, 2}; Y in {0, 4}

Hyper Arc consistency

It's basically Arc consistency, but the constriant C works on more than two variables.

Example:

2X + 3Y < Z; X in {1..10}; Y in {1..10}; Z in {1..10};

This CSP is consistent, but not arc consistent The equivalent CPS with HAC is:

2X + 3Y < Z; X in {1..3}; Y in {1..2}; Z in {6..10};

Hierarchy of consistency

Hyper-arc Consistency (GAC) ↓ (implies)

Arc Consistency (AC) ↓ (implies)

Bounds Consistency (BC) ↓ (implies)

Node consistency (NC) it doesn't imply if CSP has solution.

Consistency (has solutions)

- n = number variables
- c = number of constraints
- d = maximum domain
- m = number of variables for the constraint

GAC is the most computationally complex but has a strong propagation. Computational complexity is: $O(ncmd^{m+1})$

AC is a middle ground between GAC and BC, but still has a strong propagation. Computational complexity is: $O(ncd^3)$

BC is the simplest in terms of computational complexity, but has a weak propagation. Can be implemented in constat computational complexity -> O(1)

NC is the weakest in terms of consistency propagation, and has a computational complexity of: O(cd) where c = $|C|$, d = max_i $|Di|$.

Consistency is just a decisional problem, we need to decide whether a CSP has solutions or not.

Path consistency

A CSP *P* is *normalized* if for each pair of variables X and Y (order does not matter) there is at most one binary constraint including them in the *P*.

If *P* is normalized, let us denote by C(X, Y) if such constraint exists, otherwise we can define it as the cartesian product C(X, Y) = Dx x Dy

A CSP *P* is *standard* if for each, unordered, pair of variables X and Y there is exactly one binary constraints on them C(X, Y) in *P*.

Let's take a look at an example:

< X + Y < 5, X + Y <= Z, X < Z, 2X != 3Z, X + Y != Z; De >

C1 (X, Y) C2 (X, Y, Z) C3 (X, Z) C4 (X, Z) C5 (X, Y, Z)

This CSP is not normalized since we have more than one binary constraint for X and Z. We can normalize it by replacing the first and second constraint by C1 ∩ C2.

If we add the constraint Dy x Dz, we can call it *all*, then this CSP becomes standard:

〈 X + Y < 5, (X < Z ∧ 2X != 3Z), *all(Y, Z)*, X + Y < Z; De 〉

C1 (X, Y) C2 (X, Z) C3 (Y, Z) C4 (X, Y, Z)

Any CSP can be transformed into a standard and equivalent CSP. During this rewriting phase we can detect unsatisfiability by forcing arc consistency

< X + Y = 0, X - Y = 0; X in {-1, 1}, Y in {-1, 1} >

this is arc consistent but not standard

< (X + Y = 0 ∧ X - Y = 0); X in {-1, 1}; Y in {-1, 0, 1} >

this is standard but not arc consistent.

Path consistency

A standard CSP is path consistent if for each triple of variables X, Y, Z it holds that:

$C(X, Z) \subseteq C(X, Y)\ C(Y,Z)$

If whenever $(a, b) \in C(X, Z)$ there is a $c \in Dy$ such that $(a, c) \in C(X, Y)$ and $(c, b) \in C(Y, Z)$

Let's look at an example:

< X != Y, Y != Z, X != Z; X in 0..1, Y in 0..1, Z in 0..1 >

Then $C(X, Z) = C(X, Y) = C(Y, Z) = \{(0, 1), (1, 0)\}$

Observe that $C(X, Y)C(Y, Z) = \{(0, 0), (1, 1)\}$

$C(X, Z) = \{(0, 1), (1, 0)\} \subseteq \{(0, 0), (1, 1)\} = C(X, Y)\ C(Y,Z)$

this is not path consistent.

Path consistency has a computational complexity of:

$O(vd\ v\ (v - 1)(v - 2)\ d^4) = O(v^4\ d^5)$

Is very costly for real problem application.

Take a look at slide 8 and 9 on MiniZinc